

Java Concurrency

Threads, Synchronisation & the java.util.concurrent Toolkit

March 2026

1. Why Concurrency Matters

Modern hardware ships with multiple CPU cores, yet a single-threaded program can only use one at a time. Concurrency lets a Java program make progress on several tasks simultaneously — whether by truly running in parallel on multiple cores or by interleaving work on a single core while waiting for I/O.

Java has supported threads since version 1.0 and has grown a rich ecosystem of higher-level abstractions. This document progresses from the raw Thread API through synchronisation primitives up to the java.util.concurrent framework, and closes with common concurrency patterns.

Core concepts

- Thread — lightweight unit of execution within a JVM process
- Race condition — two threads access shared state concurrently with at least one write
- Deadlock — two threads each hold a lock the other needs
- Starvation — a thread can never acquire a resource because others always get there first
- Visibility — changes made by one thread may not be seen by another without proper synchronisation

2. Creating and Starting Threads

2.1 Extending Thread

The simplest approach: subclass Thread and override run().

```
public class CounterThread extends Thread {
    private final int id;
    public CounterThread(int id) { this.id = id; }
    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("Thread-" + id + " count: " + i);
        }
    }
}
```

```
// Usage
Thread t1 = new CounterThread(1);
Thread t2 = new CounterThread(2);
t1.start(); // start() creates OS thread and calls run()
t2.start();
t1.join(); // main waits for t1 to finish
t2.join();
```

Prefer Runnable or Callable over extending Thread — it separates the task from the execution mechanism.

2.2 Implementing Runnable

Runnable is a functional interface, so it works neatly with lambdas.

```
Runnable task = () -> {
    System.out.println("Running in: " + Thread.currentThread().getName());
};
Thread t = new Thread(task, "worker-1");
t.start();
```

2.3 Callable and Future

Callable<V> is like Runnable but can return a value and throw checked exceptions. Pair it with Future<V> to retrieve the result.

```
import java.util.concurrent.*;

Callable<Integer> heavyCalc = () -> {
    Thread.sleep(500); // simulate work
    return 42;
};

ExecutorService exec = Executors.newSingleThreadExecutor();
Future<Integer> future = exec.submit(heavyCalc);
// ... do other work ...

int result = future.get(); // blocks until done
System.out.println("Result: " + result);
exec.shutdown();
```

3. Synchronisation Primitives

3.1 synchronized Keyword

Every Java object has an intrinsic monitor lock. The synchronized keyword acquires that lock before entering the block/method and releases it on exit — even if an exception is thrown.

```
public class SafeCounter {
    private int count = 0;
    // Method-level lock — locks 'this'
    public synchronized void increment() {
        count++;
    }
    // Block-level lock — finer granularity
    public void decrement() {
        synchronized (this) {
            count--;
        }
    }
    public synchronized int getCount() { return count; }
}

// Usage
SafeCounter counter = new SafeCounter();
Thread t1 = new Thread(() -> { for (int i=0;i<1000;i++)
    counter.increment(); });
Thread t2 = new Thread(() -> { for (int i=0;i<1000;i++)
    counter.decrement(); });
t1.start(); t2.start();
t1.join(); t2.join();
System.out.println(counter.getCount()); // always 0
```

Pitfall Synchronising on different objects provides NO mutual exclusion. Always lock on the same shared object.

3.2 volatile Keyword

volatile guarantees visibility: every write is immediately flushed to main memory, and every read comes from main memory. It does NOT provide atomicity for compound operations like `i++`.

```
public class StatusFlag {
    private volatile boolean running = true;
    public void stop() { running = false; } // write visible to all threads
    public void run() {
        while (running) { // always reads fresh value
            // ... do work ...
        }
        System.out.println("Stopped.");
    }
}
```

3.3 wait / notify

Low-level cooperation between threads using the monitor lock. Must be called inside a synchronized block.

```
import java.util.LinkedList;

public class BoundedQueue<T> {
    private final LinkedList<T> queue = new LinkedList<>();
    private final int capacity;

    public BoundedQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait(); // release lock & sleep
        queue.add(item);
        notifyAll(); // wake waiting consumers
    }

    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) wait();
        T item = queue.poll();
        notifyAll(); // wake waiting producers
        return item;
    }
}
```

```

    }
}

```

Always call wait() inside a while loop, not an if — spurious wake-ups can occur.

4. java.util.concurrent Framework

Introduced in Java 5, the java.util.concurrent (j.u.c) package provides production-grade building blocks that replace most hand-rolled synchronisation.

4.1 Atomic Variables

AtomicInteger, AtomicLong, AtomicReference, and friends offer lock-free thread-safe operations via Compare-And-Swap (CAS) hardware instructions.

```

import java.util.concurrent.atomic.*;

public class AtomicCounter {
    private final AtomicInteger value = new AtomicInteger(0);

    public void increment() { value.incrementAndGet(); }
    public void decrement() { value.decrementAndGet(); }
    public int get() { return value.get(); }

    // Compare-and-swap: set to newVal only if current == expected
    public boolean compareAndSet(int expected, int newVal) {
        return value.compareAndSet(expected, newVal);
    }
}

```

4.2 ReentrantLock

A more flexible alternative to synchronized. Supports try-locking, timed locking, interruptible locking, and fairness.

```

import java.util.concurrent.locks.*;

public class BankAccount {
    private double balance;

    private final ReentrantLock lock = new ReentrantLock();

    public BankAccount(double balance) { this.balance = balance; }

    public void transfer(BankAccount target, double amount) {
        lock.lock();

        try {

```

```

    if (balance < amount) throw new IllegalStateException("Insufficient
    funds");
    balance -= amount;
    target.balance += amount;
    } finally {
    lock.unlock(); // ALWAYS unlock in finally
    }
    }
    // Try-lock variant — non-blocking
    public boolean tryTransfer(BankAccount target, double amount) {
    if (lock.tryLock()) {
    try { balance -= amount; target.balance += amount; return true; }
    finally { lock.unlock(); }
    }
    return false; // lock not available right now
    }
    }

```

4.3 ReadWriteLock

Allows concurrent reads but exclusive writes — ideal for read-heavy caches.

```

import java.util.concurrent.locks.*;
import java.util.*;

public class ReadWriteCache<K, V> {
    private final Map<K, V> map = new HashMap<>();
    private final ReadWriteLock rwl = new ReentrantReadWriteLock();
    private final Lock readLock = rwl.readLock();
    private final Lock writeLock = rwl.writeLock();

    public V get(K key) {
    readLock.lock(); // multiple threads can hold this simultaneously
    try { return map.get(key); }
    finally { readLock.unlock(); }
    }
}

```

```

public void put(K key, V value) {
    writeLock.lock(); // exclusive — blocks all readers & writers
    try { map.put(key, value); }
    finally { writeLock.unlock(); }
}
}

```

4.4 ExecutorService & Thread Pools

Creating a new Thread per task is expensive. ExecutorService manages a pool of reusable threads and queues submitted tasks.

```

import java.util.concurrent.*;

public class ThreadPoolDemo {
    public static void main(String[] args) throws Exception {
        // Fixed pool — good for CPU-bound tasks (size = # cores)
        ExecutorService fixed = Executors.newFixedThreadPool(4);
        // Cached pool — unbounded, reuses idle threads; good for short
        // I/O tasks
        ExecutorService cached = Executors.newCachedThreadPool();
        // Scheduled pool — run tasks after a delay or periodically
        ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(2);
        // Submit tasks
        List<Future<Integer>> futures = new ArrayList<>();
        for (int i = 0; i < 8; i++) {
            final int taskId = i;
            futures.add(fixed.submit(() -> {
                System.out.println("Task " + taskId + " on " + Thread.currentThread().getName());
                return taskId * taskId;
            }));
        }
        // Collect results
        for (Future<Integer> f : futures) {
            System.out.println("Result: " + f.get());
        }
    }
}

```

```

    }
    // Graceful shutdown
    fixed.shutdown();
    fixed.awaitTermination(10, TimeUnit.SECONDS);
  }
}

```

4.5 CompletableFuture

Java 8 introduced `CompletableFuture` — a composable, non-blocking way to chain async operations.

```

import java.util.concurrent.*;

CompletableFuture
    .supplyAsync(() -> fetchUserFromDb(42)) // runs in ForkJoinPool
    .thenApply(user -> enrichWithProfile(user)) // transform result
    .thenApply(user -> sendWelcomeEmail(user))
    .thenAccept(res -> System.out.println("Done: " + res))
    .exceptionally(ex -> { System.err.println(ex); return null; });

// Combine two independent async calls

CompletableFuture<String> f1 = CompletableFuture.supplyAsync(()
-> "Hello");

CompletableFuture<String> f2 = CompletableFuture.supplyAsync(()
-> " World");

String result = f1.thenCombine(f2, String::concat).get();

System.out.println(result); // Hello World

```

4.6 Concurrent Collections

Attribute	Detail
ConcurrentHashMap	Thread-safe HashMap; segment-level locking (no full lock). Prefer over <code>Collect</code>
CopyOnWriteArrayList	Reads are lock-free; writes create a fresh copy. Best for read-heavy, rarely-mut
BlockingQueue	<code>LinkedBlockingQueue</code> / <code>ArrayBlockingQueue</code> for producer-consumer pipelines;
ConcurrentLinkedQueue	Lock-free non-blocking FIFO queue for high-throughput task queues.
ConcurrentSkipListMap	Sorted thread-safe map (like <code>TreeMap</code>); $O(\log n)$ operations without global loc

```

import java.util.concurrent.*;

// Producer-Consumer with BlockingQueue

```



```

BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);
Thread producer = new Thread(() -> {
    try {
        for (int i = 0; i < 5; i++) {
            queue.put("item-" + i); // blocks if full
            System.out.println("Produced: item-" + i);
        }
        queue.put("DONE"); // sentinel value
    } catch (InterruptedException e) { Thread.currentThread().interrupt();
    }
});
Thread consumer = new Thread(() -> {
    try {
        String item;
        while (!(item = queue.take()).equals("DONE")) { // blocks if empty
            System.out.println("Consumed: " + item);
        }
    } catch (InterruptedException e) { Thread.currentThread().interrupt();
    }
});
producer.start(); consumer.start();
producer.join(); consumer.join();

```

5. Synchronisation Utilities

5.1 CountDownLatch

A one-time gate: threads wait at `await()` until the count reaches zero. Useful for waiting for multiple parallel tasks to finish.

```

import java.util.concurrent.*;

int workers = 3;

CountDownLatch latch = new CountDownLatch(workers);

for (int i = 0; i < workers; i++) {
    final int id = i;
    new Thread(() -> {

```

```

System.out.println("Worker " + id + " done");
latch.countDown(); // decrement counter
}).start();
}

latch.await(); // main thread waits until count == 0
System.out.println("All workers finished");

```

5.2 CyclicBarrier

Like CountDownLatch but reusable: all threads must reach the barrier before any can continue. Good for phased simulations.

```

import java.util.concurrent.*;

CyclicBarrier barrier = new CyclicBarrier(3, () ->
System.out.println("--- All reached barrier, starting next phase ---")
);

for (int i = 0; i < 3; i++) {
    final int id = i;
    new Thread(() -> {
        try {
            System.out.println("Thread " + id + " phase 1 done");
            barrier.await(); // wait for others
            System.out.println("Thread " + id + " phase 2 done");
            barrier.await(); // reuse for second phase
        } catch (Exception e) { e.printStackTrace(); }
    }).start();
}

```

5.3 Semaphore

Controls access to a limited pool of resources (e.g., database connections, rate-limiting API calls).

```

import java.util.concurrent.*;

// Allow at most 3 concurrent database connections
Semaphore sem = new Semaphore(3);

Runnable dbTask = () -> {

```

```

try {
    sem.acquire(); // blocks if 0 permits available
    System.out.println(Thread.currentThread().getName() + " ac-
    quired connection");
    Thread.sleep(200); // simulate DB work
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
} finally {
    sem.release(); // always release
    System.out.println(Thread.currentThread().getName() + " released
    connection");
}
};

// 6 threads compete for 3 permits
for (int i = 0; i < 6; i++) new Thread(dbTask, "T" + i).start();

```

5.4 Phaser

A flexible, reusable synchronisation barrier supporting dynamic registration of parties.

```

import java.util.concurrent.*;

Phaser phaser = new Phaser(1); // register main thread
for (int i = 0; i < 3; i++) {
    phaser.register(); // register each worker
    final int id = i;
    new Thread(() -> {
        System.out.println("Worker " + id + " phase 0");
        phaser.arriveAndAwaitAdvance(); // barrier
        System.out.println("Worker " + id + " phase 1");
        phaser.arriveAndDeregister(); // done
    }).start();
}

phaser.arriveAndAwaitAdvance(); // main releases workers
phaser.arriveAndDeregister(); // main done

```

6. Virtual Threads (Java 21+)

Project Loom introduced Virtual Threads in Java 21 as a standard feature. Virtual threads are lightweight threads managed by the JVM rather than the OS, making it practical to create millions of them.

Key differences from platform threads

- Platform thread — 1:1 mapping to an OS thread; stack ~1 MB; ~thousands feasible
- Virtual thread — M:N mapping onto a small pool of carrier OS threads; stack grows on demand; ~millions feasible
- Virtual threads shine for I/O-bound workloads (blocking DB calls, HTTP requests)
- CPU-bound tasks get no benefit — use ForkJoinPool / parallel streams instead

```
// Creating virtual threads
```

```
Thread vt = Thread.ofVirtual().name("vt-1").start(() ->
```

```
System.out.println("Hello from " + Thread.currentThread())
```

```
);
```

```
vt.join();
```

```
// Virtual thread executor — simplest pattern for HTTP servers etc.
```

```
try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
```

```
    for (int i = 0; i < 10_000; i++) {
```

```
        final int id = i;
```

```
        exec.submit(() -> {
```

```
            Thread.sleep(Duration.ofMillis(100)); // blocks carrier, not OS thread
```

```
            return id;
```

```
        });
```

```
    }
```

```
} // auto-shutdown via AutoCloseable
```

```
// Structured concurrency (JDK 21 preview -> standard in 25)
```

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
```

```
    Future<String> user = scope.fork(() -> fetchUser(42));
```

```

Future<String> orders = scope.fork(() -> fetchOrders(42));
scope.join().throwIfFailed();
System.out.println(user.get() + " | " + orders.get());
}

```

Tip Avoid pinning virtual threads: do not call synchronized on a monitor inside a virtual thread if the critical section does I/O. Use ReentrantLock instead — it unmounts the virtual thread while waiting.

7. Common Concurrency Patterns

7.1 Thread-Per-Request

Assign one thread (or one virtual thread) to each incoming request. Simple and maps well to blocking I/O. The classic model for Java web servers.

```

ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor();
// Simulate a tiny HTTP server loop
while (true) {
    Socket socket = serverSocket.accept(); // blocking accept
    pool.submit(() -> handleRequest(socket));
}

```

7.2 Fork/Join

Divide-and-conquer parallelism: split a problem recursively until small enough, solve in parallel, join results. ForkJoinPool uses work-stealing to keep all cores busy.

```

import java.util.concurrent.*;

public class ParallelSum extends RecursiveTask<Long> {
    private static final int THRESHOLD = 1_000;
    private final long[] arr;
    private final int start, end;

    ParallelSum(long[] arr, int start, int end) {
        this.arr = arr; this.start = start; this.end = end;
    }

    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {

```

```

    long sum = 0;
    for (int i = start; i < end; i++) sum += arr[i];
    return sum;
}

int mid = (start + end) / 2;
ParallelSum left = new ParallelSum(arr, start, mid);
ParallelSum right = new ParallelSum(arr, mid, end);
left.fork(); // async
return right.compute() + left.join(); // sync
}
}

// Usage
long[] data = new long[1_000_000];
Arrays.fill(data, 1L);
ForkJoinPool pool = new ForkJoinPool();
long total = pool.invoke(new ParallelSum(data, 0, data.length));
System.out.println("Sum: " + total); // 1000000

```

7.3 Pipeline (Producer-Consumer Chain)

Connect stages via BlockingQueues. Each stage runs in its own thread, processing items as they arrive.

```

BlockingQueue<String> raw = new LinkedBlockingQueue<>(50);
BlockingQueue<String> processed = new LinkedBlockingQueue<>(50);

// Stage 1: produce raw data
Thread producer = new Thread(() -> {
    try {
        for (int i=0; i<10; i++) { raw.put("item-"+i); }
        raw.put("END");
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});

// Stage 2: transform

```

```

Thread transformer = new Thread(() -> {
    try {
        String item;
        while (!(item = raw.take()).equals("END")) {
            processed.put(item.toUpperCase());
        }
        processed.put("END");
    } catch (InterruptedException e) { Thread.currentThread().interrupt();
    }
});

// Stage 3: consume
Thread consumer = new Thread(() -> {
    try {
        String item;
        while (!(item = processed.take()).equals("END")) {
            System.out.println("Consumed: " + item);
        }
    } catch (InterruptedException e) { Thread.currentThread().interrupt();
    }
});

producer.start(); transformer.start(); consumer.start();
producer.join(); transformer.join(); consumer.join();

```

8. Quick-Reference Summary

Use this table to quickly pick the right tool for a concurrency scenario.

Tool / Concept	Best For	Watch Out For
synchronized	Simple mutual exclusion on shared state	Deadlock if lock ordering not consistent
volatile	Single flag visibility across threads	Does not atomise compound ops (i++)
AtomicInteger etc.	Lock-free counters & CAS operations	Only single-variable; no multi-step atomicity
ReentrantLock	Timed, tryLock, fair locking	Must unlock in finally block
ReadWriteLock	Read-heavy caches	Write starvation under heavy read load
ExecutorService	Thread-pool task execution	Always shutdown(); avoid ThreadLocal leaks
CompletableFuture	Async pipelines without blocking threads	Unhandled exceptions swallowed silently
BlockingQueue	Producer-consumer pipelines	Choose bounded capacity to avoid OOM
CountDownLatch	One-shot wait for N events	Non-reusable; use CyclicBarrier if needed again

Semaphore	Limit concurrent access to resource pool	Forget <code>release()</code> in <code>finally</code> = permit leak
Virtual Threads	Massive I/O concurrency (Java 21+)	Avoid synchronized + blocking I/O (pinning)
ForkJoinPool	Recursive divide-and-conquer tasks	Task granularity too fine = overhead

9. Further Reading

Java Concurrency in Practice — Brian Goetz et al. (2006) — the definitive reference

The Art of Multiprocessor Programming — Herlihy & Shavit (2nd ed. 2020)

JEP 444: Virtual Threads — <https://openjdk.org/jeps/444>

JEP 453: Structured Concurrency — <https://openjdk.org/jeps/453>

Baeldung Java Concurrency series — <https://www.baeldung.com/java-concurrency>